

21. Modifying the Document (DOM badalna)

DOM modification se pages "live" bante hain. Naye elements bana kar page me daal sakte ho.

Element banana

js

```
let div = document.createElement('div');           // element node
let text = document.createTextNode('Here I am');    // text node (kam use hota ha
```

Example (message div):

js

```
let div = document.createElement('div');
div.className = "alert";
div.innerHTML = "<strong>Hi there!</strong> Important message.";
document.body.append(div); // ye step zaruri, warna page me dikhega nahi
```

Element bana lena kaafi nahi, use **insert** karna padta hai.

Insertion methods (modern)

Method	Kaam
<code>node.append(...)</code>	node ke andar, end me
<code>node.prepend(...)</code>	node ke andar, shuru me
<code>node.before(...)</code>	node ke pehle
<code>node.after(...)</code>	node ke baad
<code>node.replaceWith(...)</code>	node ko replace kar de

- Arguments: DOM nodes ya strings (kitne bhi).
- **Strings "as text" insert hoti hain**, HTML nahi (`<`, `>`) escape ho jaate hain), `textContent` jaisa safe.

js

```
div.before('<p>Hello</p>', document.createElement('hr'));  
// "<p>Hello</p>" literally text dikhega
```

`insertAdjacentHTML(where, html)` (HTML string daalne ke liye)

`where` ke 4 options:

- `"beforebegin"`: elem ke pehle
- `"afterbegin"`: elem ke andar, shuru me
- `"beforeend"`: elem ke andar, end me
- `"afterend"`: elem ke baad

js

```
div.insertAdjacentHTML('beforebegin', '<p>Hello</p>');  
div.insertAdjacentHTML('afterend', '<p>Bye</p>');
```

Iske bhai `insertAdjacentText` aur `insertAdjacentElement` bhi hain, par kam use hote hain.

Remove aur Move

- `node.remove()`: hata do.
- Kisi element ko **move** karna ho to pehle remove karne ki zarurat nahi. Insert methods use **purani jagah se khud hata dete hain**:

js

```
second.after(first); // first ab second ke baad, remove ki zarurat nahi
```

Clone: `cloneNode`

js

```
let div2 = div.cloneNode(true); // deep clone (sab children ke saath)  
// cloneNode(false) = bina children ke  
div.after(div2);
```

DocumentFragment

Nodes ki list ko ek wrapper me pass karne ke liye special node. Insert karne par khud gayab ho jaata hai, sirf uske andar ke nodes insert hote hain. Aajkal kam use hota hai, array of nodes + `append(...arr)` bhi chalta hai.

Purane (old-school) methods

Purani scripts me milte hain, naye code me zarurat nahi:

- `parent.appendChild(node)`
- `parent.insertBefore(node, nextSibling)`
- `parent.replaceChild(newElem, node)`
- `parent.removeChild(node)`

`document.write`

- Page ke loading ke waqt hi chalta hai. Page load hone ke **baad** call karoge to **poora document mita deta hai**.
- Fayda: tez hai (DOM modify hi nahi karta). Par ab lagbhag koi use nahi karta.

Tasks ke jawab (short)

- `elem.append(document.createTextNode(text))` aur `elem.textContent = text` same hain (dono "as text"). `innerHTML = text` alag hai.
- Element khaali karna:

js

```
function clear(elem) { while (elem.firstChild) elem.firstChild.remove(); }  
// ya elem.innerHTML = '';
```

(Loop me `childNodes[i].remove()` galat hai, kyunki remove se collection shift ho jaata hai.)

- Do `li` beech me daalna: `one.insertAdjacentHTML('afterend', '<li>2</li><li>3</li>')`
- Table sort: rows ko `Array.from(tbody.rows).sort(...)`, phir `tbody.append(...sortedRows)`.
- `<table>` ke andar seedha text ("aaa") galat HTML hai, browser use table ke **bahar** **pehle** rakh deta hai, isliye `table.remove()` par bhi wo bacha rehta hai.

Yaad rakho

`createElement` -> set properties -> `append/prepend/before/after`. HTML string ho to `insertAdjacentHTML`. User ka text ho to `textContent` ya strings wale methods (safe).